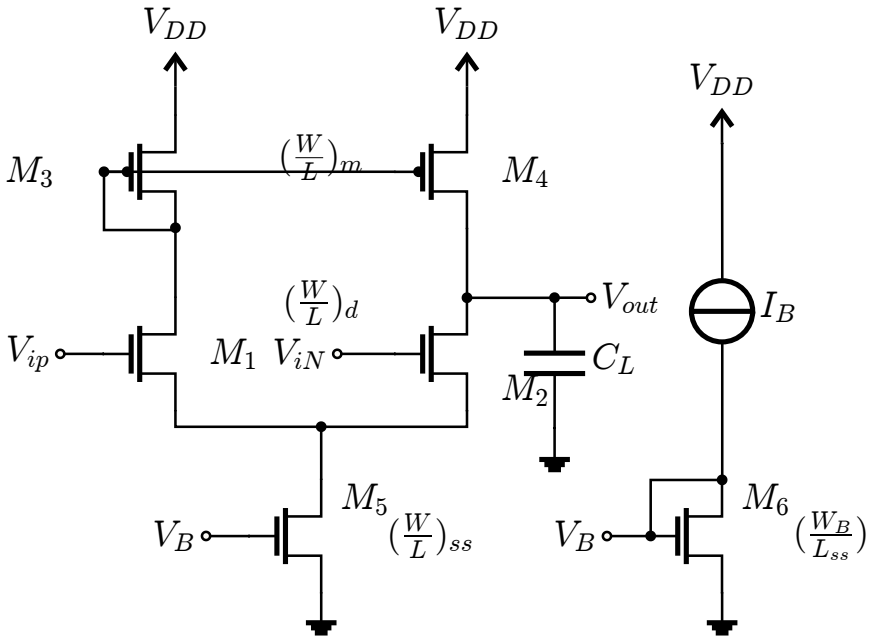
Analog Design using Agentic AI

OTA-5T Design Using AntiGravity + Gemini

1. Setup & Tools

- **LLM**: Gemini Pro for Part 3, Gemini Flash for Parts 1 & 2.
- **Simulator**: LTspice, Ngspice, or PyTorchSim (under development).
- **Automation Scripts & Circuit Wrappers**: Python.
- **Analog Optimizer**: Genetic Algorithm using SciPy.
- **Small ADT**: For LUTs utilizing the `LUT_FinderAgent` (developed in Python).
- **Technology**: GF-180nm, 3.3V Supply.

2. 5T OTA Variables & Target Specifications



Feature Variables (Inputs):

These are the design parameters we need to size:
$\vec{x} = [W_d, L_d, W_m, L_m, W_{ss}, L_{ss}, I_b, V_{ICM}]$

Target Variables (Outputs):

We usually want certain outputs or target specs ($\vec{y}_{TG}$) from the circuit:
$[A_v, BW, UGF, CMRR, PM, P_W, SR, V_{ICM,min}, V_{ICM,max}, V_{out,min}, V_{out,max}, V_{out,CM}]$

The Design Objectives:

$$A_v \geq A_{v,TG}$$
$$BW \geq BW_{TG}$$
$$UGF \geq UGF_{TG}$$
$$CMRR \geq CMRR_{TG}$$

$$PM \geq PM_{TG}$$
$$P_W \leq P_{W,TG}$$
$$SR \geq SR_{TG}$$
$$V_{ICM,min} \leq V_{ICM,min,TG}$$
$$V_{ICM,max} \geq V_{ICM,max,TG}$$
$$V_{out,min} \leq V_{out,min,TG}$$
$$V_{out,max} \geq V_{out,max,TG}$$
$$V_{out,CM} = V_{out,CM,TG}$$

3. Optimizing Simulation Speed

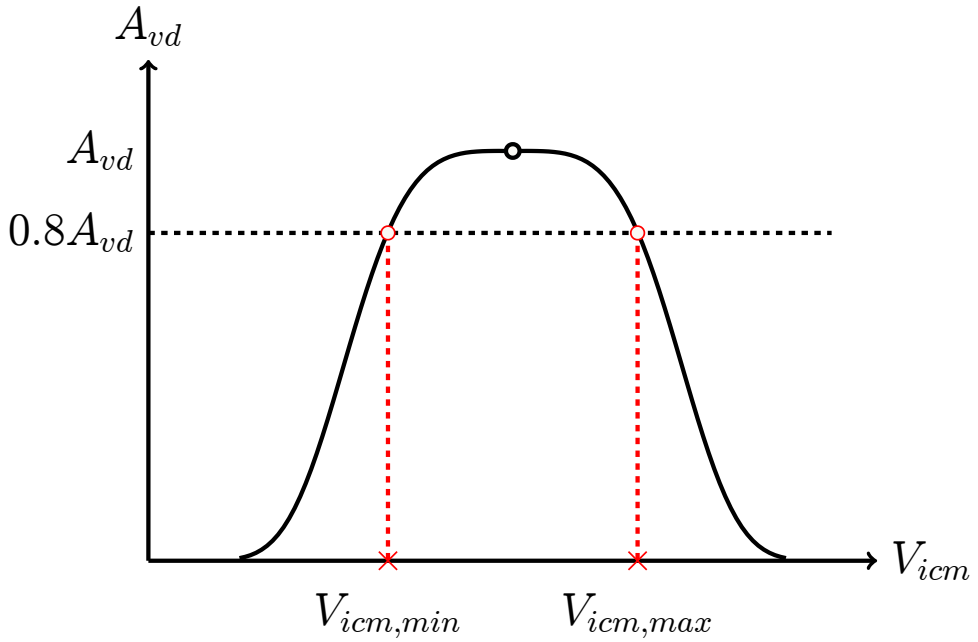
We will later discover that using this exact version of the target variables will be problematic.

To get $V_{ICM,min}$ and $V_{ICM,max}$, we would need to sweep V_{ICM} and perform an AC analysis per step to get a waveform for A_v vs. V_{ICM}. Then, from this, we have to find points where $A_v(V_{ICM}) = 0.8A_v$.
The solution to this yields two values: $V_{ICM,min}$ and $V_{ICM,max}$.

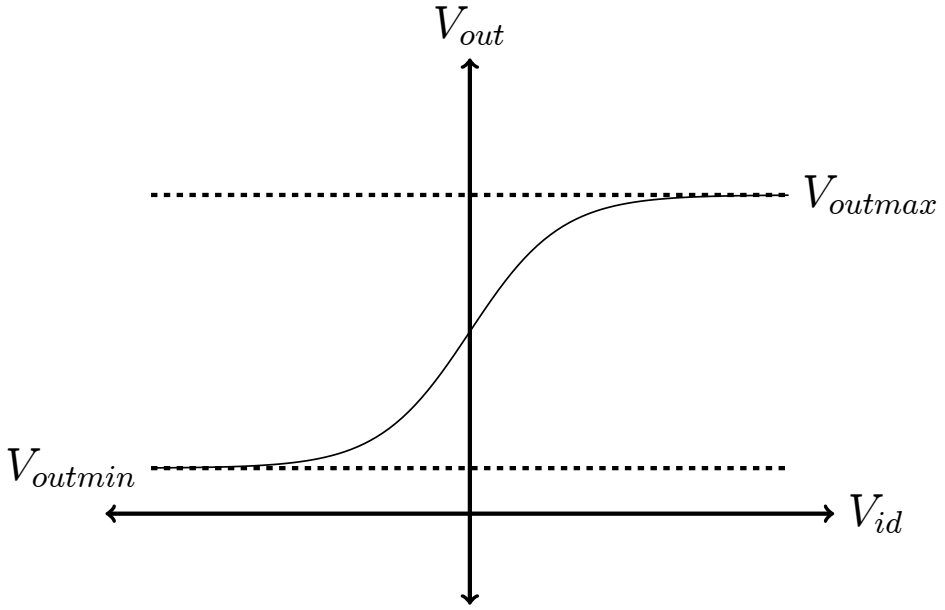
Instead of running a massive sweep inside an optimization loop (which is a disaster for simulation speed), we modify the condition. Measuring differential gain at a certain input value only requires a single AC simulation. Thus, we directly use:

$$A_v(V_{ICM} = V_{ICM,min,TG}) \geq 0.8A_{v,TG}$$
$$A_v(V_{ICM} = V_{ICM,max,TG}) \geq 0.8A_{v,TG}$$

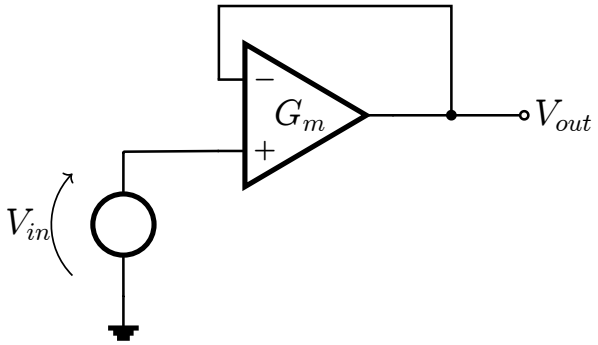
This boosts our simulation speed significantly.



Similarly, to get $V_{out,min}$ and $V_{out,max}$, you traditionally have to do a DC sweep over the differential input and plot the output waveform & measure its maxima and minima



Instead of doing a DC sweep, we configure the OTA as a buffer.
In a buffer configuration, the circuit will do its best to make $V_{out} \approx V_{in}$.



- If we use $V_{in} = V_{out,min,TG}$, the circuit will output $V_{out,min,actual}$.

$$V_{out,min,err} = V_{out,actual} - V_{out,min,TG}$$

- If we use $V_{in} = V_{out,max,TG}$, the circuit will output $V_{out,max,actual}$.

$$V_{out,max,err} = V_{out,actual} - V_{out,max,TG}$$

These errors can simply be measured using a single DC Operating Point simulation.

Modified Target Specs:

$$A_v(V_{ICM} = V_{ICM,min,TG}) \geq 0.8A_{v,TG}$$
$$A_v(V_{ICM} = V_{ICM,max,TG}) \geq 0.8A_{v,TG}$$
$$V_{out,min,err}(V_{in} = V_{out,min,TG}) \leq 0$$
$$V_{out,max,err}(V_{in} = V_{out,max,TG}) \geq 0$$

Full New Target Variables Vector:

$$\begin{aligned}\vec{y} = [&A_v, BW, UGF, CMRR, PM, P_W, SR,\\ &Av(V_{ICM} = V_{ICMminTG}), Av(V_{ICM} = V_{ICMmaxTG}),\\ &Voutminerr(Vin = V_{outminTG}), Voutmaxerr(Vin = V_{outmaxTG}),\\ &V_{OUTCM}]\end{aligned}$$

Full Modified Target Specs:

$$
\begin{aligned}
Av &\geq A_{v,TG} \\
BW &\geq BW_{TG} \\
UGF &\geq UGF_{TG} \\
CMRR &\geq CMRR_{TG} \\
PM &\geq PM_{TG} \\
P_w &\leq P_{wTG} \\
SR &\geq SR_{TG} \\
Av(V_{ICM} = V_{ICMminTG}) &\geq 0.8A_{vTG} \\
Av(V_{ICM} = V_{ICMmaxTG}) &\geq 0.8A_{vTG} \\
Voutminerr(Vin = V_{outminTG}) &\leq 0 \\
Voutmaxerr(Vin = V_{outmaxTG}) &\geq 0 \\
V_{outCM} &= V_{outCMTG}
\end{aligned}
$$

4. The 5T OTA as a Mathematical Function

Neither Python nor an LLM model can access the circuit as a schematic topology directly. We use a Python function called a **circuit wrapper**.

A wrapper is a Python script that takes the design features
(e.g., $L_d, W_d, L_m, W_m, L_{ss}, W_{ss}, I_b$) as inputs, calls the simulator, and outputs the results back.
It converts the 5T OTA into a simple mathematical function:

$$\vec{y} = \vec{f}(\vec{x})$$

To Python and the LLM, the circuit is simply a function mapping an input vector of numbers to an output vector.

5. The Design Plan

We can't just give the LLM our targets ($\vec{y}_{TG}$) and ask it to find the best possible $\vec{x}_{opt}$ such that $\vec{f}(\vec{x}_{opt}) \approx \vec{y}_{TG}$. Even with a powerful LLM, treating it like a "spice monkey" forces it to guess iteratively. This consumes the limited token quota in the session and usually fails.

Instead of treating the LLM as an iterator, we treat it as a reasoning model (Junior Designer) whom we teach the design methodology via a `.md` file.

Step 1: Initial Sizing via g_m/I_D

We write a `.md` file explaining the step-by-step g_m/I_D methodology to obtain an initial design point ($\vec{x}_{init}$). LLMs are reasoning models, not calculators. To solve specific parameter lookups, we utilize the `LUT_FinderAgent`. For example, if the LLM determines that M_1 requires a $g_m/I_D = 10$ and $L_1 = 1\mu m$, it will query the `LUT_FinderAgent` to obtain the exact transistor width, bypassing its own computational limits.

When simulating $\vec{x}_{init}$, the output $\vec{y}_{init}$ will be close to $\vec{y}_{TG}$, but rarely perfect on the first try.

A Junior Designer will not know what to do, the LLM won't know what to do too, so in order to not consume our valuable tokens & simulations (to prevent the LLM from going back to acting like an iterator or a spice monkey) we use a Python tool called the optimizer.

(mostly Genetic Algorithm optimizer, because analog design is a non-convex optimization problem GA is most suited for this task).

Note : Some people use particle Swarm/Bayesian optimizers

Step 2: The Optimizer

Since the LLM shouldn't waste tokens doing trial and error, we pass $\vec{x}_{init}$ to a Python optimizer. We mostly use a Genetic Algorithm (GA) because analog design is a non-convex optimization problem, though

Particle Swarm or Bayesian optimizers are also valid.
The optimizer treats the design as a math problem: find $\vec{x}_{opt}$ such that $\vec{f}(\vec{x}_{opt}) \approx \vec{y}_{TG}$. To do this, we define a loss function L_{tot}.

Loss Functions:

1. For specs to maximize ($A_v, BW, UGF, CMRR, PM, SR$):
If $\vec{y}_A$ is the subset of outputs we want to maximize, and $\vec{y}_{A,TG}$ are the targets:

$$L_{A_k} = \text{ReLU}\left(\frac{y_{A_k,TG} - y_{A_k}}{|y_{A_k,TG}|}\right)$$

$$L_A = \sum_{k=1}^{n} L_{A_k}$$

$$L_A = \sum_{k=1}^{l_A} \text{ReLU}\left(\frac{y_{A_k,TG} - y_{A_k}}{|y_{A_k,TG}|}\right)$$

(If the target is achieved, $y_{A_k} \geq y_{A_k,TG}$, the inner term is negative, and ReLU outputs 0 loss).

2. For specs to minimize ($P_W, V_{out,min,err}, V_{out,max,err}$):

$$L_{B_k} = \text{ReLU}\left(\frac{y_{B_k} - y_{B_k,TG}}{|y_{B_k,TG}|}\right)$$

$$L_B = \sum_{k=1}^{l_B} L_{B_k}$$

$$L_B = \sum_{k=1}^{l_B} \text{ReLU}\left(\frac{y_{B_k,TG} - y_{B_k}}{|y_{B_k,TG}|}\right)$$

3. For specs to equate ($V_{out,CM}$):

$$L_{C_k} = \frac{|y_{C_k} - y_{C_k,TG}|}{|y_{C_k,TG}|}$$

$$L_C = \sum_{k=1}^{l_C} L_{C_k}$$

$$L_C = \sum_{k=1}^{l_C} \text{ReLU}\left(\frac{y_{C_k,TG} - y_{C_k}}{|y_{C_k,TG}|}\right)$$

Total Loss:

$$L_{tot} = L_A + L_B + L_C$$

The GA minimizes this total loss:

$$\min_{\vec{x}} L_{tot}(\vec{f}(\vec{x}) - \vec{y}_{TG})$$

By giving the optimizer a strong initial design point ($\vec{x}_{init}$) and a bounded search space, it acts like searching for a key in a specific drawer rather than blindly wandering the neighborhood.

One might think if the optimizer can do this on his own & find the optimal sizing, why not just discard the LLM & focus on the optimizer alone!!

For 2 important reasons, the first is you can't guarantee your loss will always go down overall & the second is even if your loss is decaying & you are converging, it might take you infinite iterations to converge.

In real world analog design, you can't do an infinite number of simulations on a single circuit, that defeats the whole concept of design.

So how do we help the optimizer?!

By giving it a good initial design point & a limited/bounded space to search for the design point in.

Think of it like that if I tell you to search for a key in your closet it's easy, if I ask you to search in your room a little harder but if I ask you to search for it in the house or in the entire neighborhood it's almost impossible.

This is why the optimizer can't do it alone.

The Optimizer is simply a search engine not a designer.

Using the optimizer alone to find the design point is similar to searching for a needle in a haystack.

The optimizer needs the LLM's logic & reasoning to get a good initial guess to start from:$\vec{x}_{init}$

The LLM needs the precision & calculations of the optimizer to smart search the design space instead of guessing & trial & error.

$$[y_1\ y_2\ y_3\ \dots\ y_m] = f(x_1, x_2, x_3, \dots, x_n)$$

$$\vec{y} = f(\vec{x})$$

$$\text{GA}[f(\vec{x}), \vec{x}_{init}, \vec{y}_{TG}, (\vec{x}_{lb}, \vec{x}_{ub}), N_{\text{max\_iter}}]$$

The GA should output: $\vec{x}_{\text{opt}}$ such that $f(\vec{x}_{\text{opt}}) \approx \vec{y}_{\text{TG}}$

Step 3: Trade-offs (The Senior Designer)

The GA is bounded by `max_sim` (maximum number of simulations/ iterations). If it fails to reach $L_{tot} = 0$, we rely on the ultimate laws of analog design: **Trade-offs**.

Navigating trade-offs requires advanced logic and reasoning, akin to a Senior Designer. This cannot be done efficiently by a mid-level model executing a step-by-step guide. We switch to our highest reasoning model (Gemini Pro) equipped with a specific `.md` file detailing circuit trade-offs for this OTA topology.

Summary of the 3-Part Architecture:

1. Initial Design Point Calculation:

Using Gemini Flash (mid-level reasoning) + `LUT_FinderAgent` guided by a g_m/I_D `.md` file.

2. Optimizer Execution:

Zero reasoning. Simply runs the GA Python script to minimize the mathematical loss function.

3. Trade-offs:

If necessary, Gemini Pro (highly advanced reasoning) evaluates the GA output and intelligently adjusts sizes based on a trade-offs `.md` file to finalize the design.